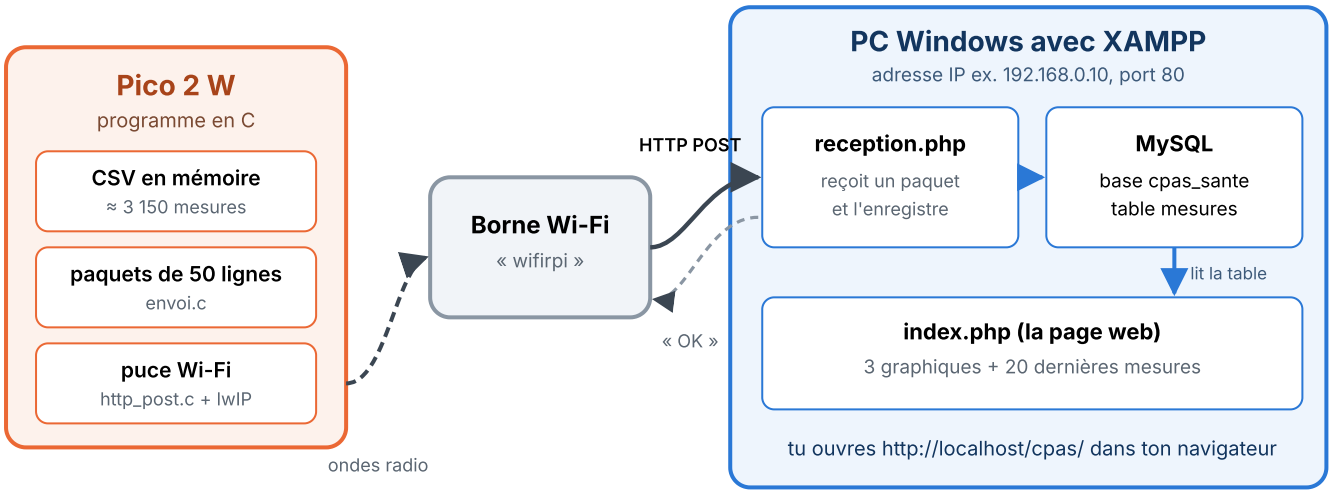
CPAS_ENVOI_SERVEUR

Version simple : comment le Pico 2 W envoie ses mesures au serveur, avec des schémas

1. Le principe

Le Pico contient un fichier **CSV de mesures** (accéléromètre + gyroscope). Quand on l'allume, il se connecte au **Wi-Fi**, découpe le CSV en **paquets de 50 lignes** et envoie chaque paquet au PC par une requête **HTTP POST**. Sur le PC, **XAMPP** reçoit les paquets avec un script PHP, les range dans une base **MySQL** et les affiche sur une **page web avec des graphiques**.



CSV → Pico (C) → Wi-Fi → reception.php → MySQL → index.php

Figure 1 : vue d'ensemble. Le Pico et le PC doivent être sur le **même réseau Wi-Fi**.

> **Comparaison.** C'est comme envoyer un gros cahier par La Poste. Le Pico arrache les pages 50 par 50, met chaque lot dans une enveloppe adressée à `192.168.0.10/cpas/reception.php` et la poste. Le facteur, c'est le Wi-Fi. À l'arrivée, `reception.php` range chaque page dans l'armoire (MySQL) et répond « OK ». Le Pico attend ce « OK » avant de poster l'enveloppe suivante.

Les fichiers qui restent

Fichier	Rôle
`pico/main.c`	connexion Wi-Fi, envoi, LED allumée à la fin
`pico/envoi.c`	découpe le CSV en paquets de 50 lignes
`pico/http_post.c`	envoie un paquet au PC (requête HTTP POST)
`pico/config.h`	nom et mot de passe du Wi-Fi, IP du PC, chemin du script
`serveur/reception.php`	reçoit un paquet et l'écrit dans MySQL
`serveur/index.php` + `graphiques.js`	la page de résultats et ses graphiques
`serveur/creer_base.sql`	crée la base et la table (à importer une fois)

2. Avant tout : la compilation

Le Pico n'a pas de disque : on ne peut pas y copier un CSV. Au moment de la compilation, le CSV est donc **transformé en code C**. Il devient un tableau de caractères `csv_donnees[]`, rangé dans le programme.



Figure 2 : `compiler.sh` lance tout ça (les étapes sont écrites dans `CMakeLists.txt`).

3. Ce que fait le Pico (main.c)

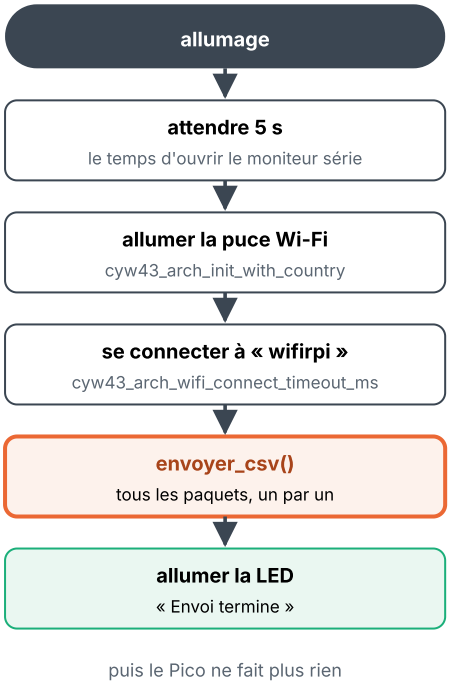


Figure 3 : tout se passe une seule fois, de haut en bas.

```
// main.c (l'essentiel)
stdio_init_all();
sleep_ms(5000);

cyw43_arch_init_with_country(
    CYW43_COUNTRY_FRANCE);
cyw43_arch_enable_sta_mode();
cyw43_arch_wifi_connect_timeout_ms(
    WIFI_SSID, WIFI_MOT_DE_PASSE,
    CYW43_AUTH_WPA2_AES_PSK, 30000);

envoyer_csv();

cyw43_arch_gpio_put(
    CYW43_WL_GPIO_LED_PIN, 1);
while (true) sleep_ms(1000);
```

Mode « sta » (station) : le Pico se comporte comme un téléphone qui rejoint une borne. La borne lui donne une adresse IP.

Pour envoyer à nouveau, il faut **débrancher puis rebrancher** le Pico.

4. Le découpage en paquets (envoi.c)

`envoyer_csv()` lit le CSV **ligne par ligne**. Il ne garde que les lignes qui commencent par un chiffre : ce sont les mesures. Les commentaires `#…` et la ligne de titres `timestamp_ms,ax_g,…` sont sautés. Les lignes gardées sont copiées dans `paquet[]`. Dès qu'il y en a 50, le paquet part.

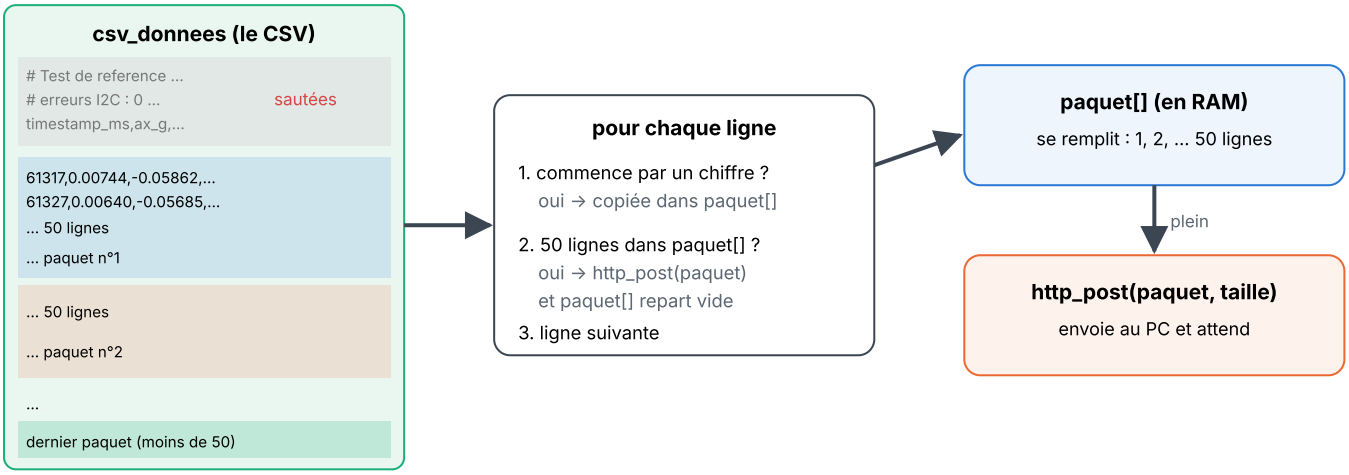


Figure 4 : environ 3 150 mesures ÷ 50 = 63 paquets. Le Pico affiche « Paquet envoye » après chacun.

```
// envoi.c : la boucle principale (simplifiée)
while (*ligne != '\0') {                          // tant qu'on n'est pas au bout du CSV
    const char *fin = strchr(ligne, '\n');        // cherche la fin de la ligne
    int n = fin - ligne + 1;
    if (ligne[0] >= '0' && ligne[0] <= '9') {     // une mesure ?
        memcpy(paquet + taille, ligne, n);        // on l'ajoute au paquet
        taille += n;  lignes++;
    }
    if (lignes == LIGNES_PAR_PAQUET) {            // paquet plein
        http_post(paquet, taille);
        taille = 0;  lignes = 0;
    }
    ligne = fin + 1;                              // ligne suivante
}
```

> **Pourquoi des paquets ?** Le CSV fait environ 190 Ko, beaucoup trop pour l'envoyer d'un coup depuis la RAM du Pico. Un paquet de 50 lignes fait environ 3 Ko.

5. L'envoi d'un paquet (http_post.c)

Le Pico parle au PC avec **TCP** (un « tuyau » fiable entre deux machines), grâce à la bibliothèque **lwIP**. Dans ce tuyau, il écrit une requête **HTTP POST**, le même format qu'un navigateur qui envoie un formulaire. lwIP fonctionne avec des **callbacks** : on lui confie deux fonctions, et il les appelle lui-même au bon moment.

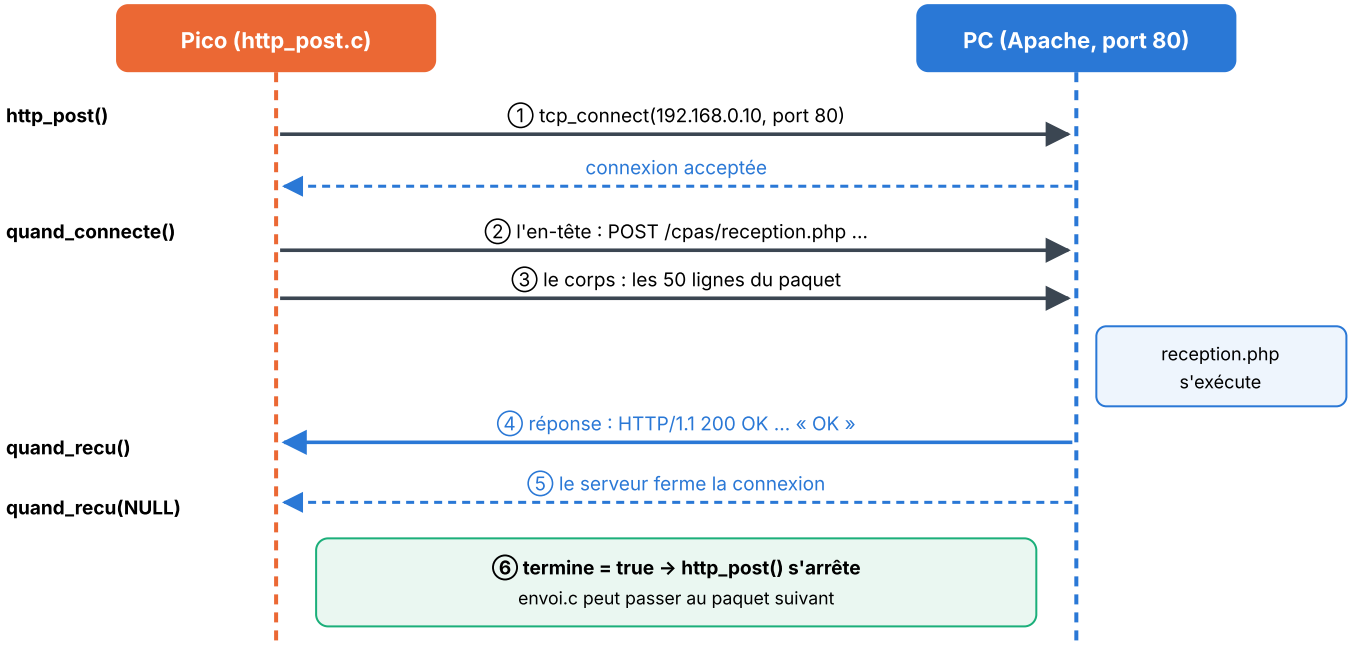


Figure 5 : la conversation pour **un** paquet. Pendant ce temps, `http_post()` attend dans une boucle `while (!termine)`.

Ce qui passe vraiment dans les ondes

```
POST /cpas/reception.php HTTP/1.0          ← l'en-tête (construit avec snprintf)
Host: 192.168.0.10
Content-Type: text/csv
Content-Length: 3050                       ← taille du corps en octets
                                           ← ligne vide : fin de l'en-tête
61317,0.00744,-0.05862,1.00485,1.00659,-0.140,0.184,-0.604     ← le corps
61327,0.00640,-0.05685,1.00363,1.00526,-0.123,0.192,-0.674
... (50 lignes)
```

Les deux callbacks : `quand_connecte()` est appelée quand le PC accepte la connexion : elle écrit l'en-tête puis le corps. `quand_recu()` est appelée quand la réponse arrive. Quand elle reçoit `NULL`, cela veut dire que le serveur a raccroché : on ferme et on met `termine = true`.
`cyw43_arch_lwip_begin()` / `end()` : lwIP tourne en arrière-plan. Ces deux lignes l'empêchent de travailler pendant qu'on prépare la connexion.

6. Côté serveur : la réception (reception.php)

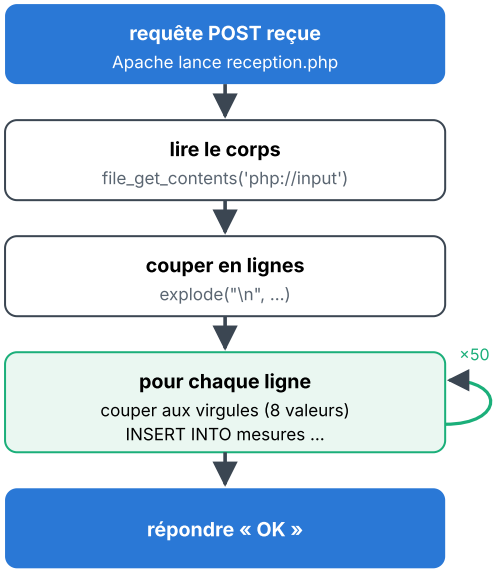


Figure 6 : le script complet.

```php
// reception.php (en entier)
require 'config.php';
$pdo = connexion();
$requete = $pdo->prepare(
  'INSERT INTO mesures (timestamp_ms,
   ax_g, ay_g, az_g, mag_g,
   gx_dps, gy_dps, gz_dps)
   VALUES (?, ?, ?, ?, ?, ?, ?, ?)');

$texte = file_get_contents('php://input');
foreach (explode("\n", trim($texte))
         as $ligne) {
  $valeurs = explode(',', trim($ligne));
  $requete->execute($valeurs);
}
echo 'OK';
```

Chaque `?` est remplacé par une des 8 valeurs de la ligne, dans l'ordre.

7. La base de données (creer_base.sql)

table mesures (base cpas_sante)	
`id`	numéro de la ligne (automatique)
`timestamp_ms`	temps depuis l'allumage du Pico (ms)
`ax_g ay_g az_g`	accélération sur x, y, z (g)
`mag_g`	accélération totale (≈ 1 g au repos)
`gx_dps gy_dps gz_dps`	rotation sur x, y, z (°/s)

On importe `creer_base.sql` **une fois** dans phpMyAdmin (onglet Importer).

Attention : ce fichier **efface la table puis la recrée vide** (`DROP TABLE`). C'est aussi la façon la plus simple de repartir de zéro avant un nouvel essai.

`config.php` se connecte à MySQL avec les réglages par défaut de XAMPP : `root`, sans mot de passe.

8. L'affichage (index.php + graphiques.js)

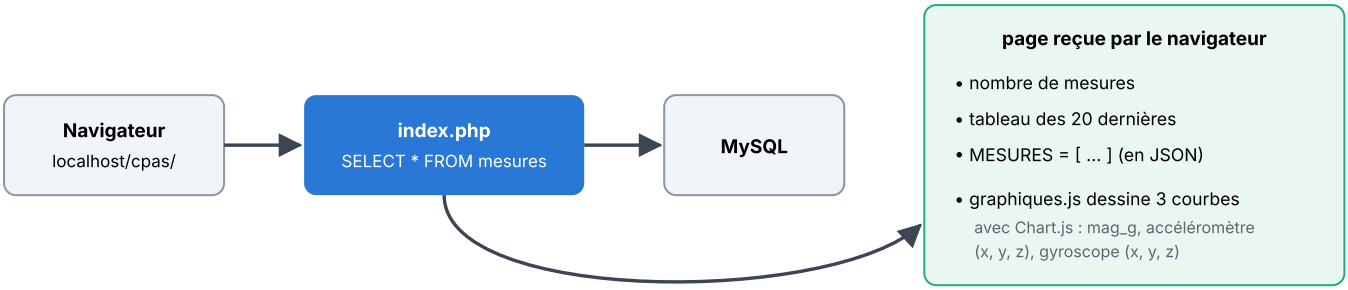


Figure 7 : PHP lit la table sur le serveur. Le navigateur reçoit une page déjà remplie, plus les mesures en JSON pour les graphiques.

9. Le trajet complet d'une mesure

#	Où	Ce qui se passe
1	PC (WSL)	`compiler.sh` met le CSV dans le programme → `cpas_envoi.uf2`, à glisser sur le Pico.
2	`main.c`	Le Pico démarre, allume le Wi-Fi et se connecte à « wifirpi ».
3	`envoi.c`	Il lit le CSV et regroupe les mesures par 50.
4	`http_post.c`	Il envoie chaque paquet au PC par une requête POST et attend la réponse.
5	`reception.php`	Le PC écrit les 50 lignes dans la table `mesures` et répond « OK ».
6	`main.c`	Une fois tous les paquets partis, la LED s'allume.
7	`index.php`	Tu ouvres `http://localhost/cpas/` et tu vois les graphiques.

10. Ce qui a été enlevé (et ce que ça change)

Enlevé	Conséquence
vérification du Wi-Fi, scan des réseaux (`wifi.c`)	si le Wi-Fi ne marche pas, le Pico essaie d'envoyer quand même, sans prévenir
délai maximum sur la réponse, codes d'erreur	si le PC ne répond pas, le Pico **reste bloqué** (la LED ne s'allume jamais)
3 essais par paquet	un paquet raté est perdu
bouton BOOTSEL pour renvoyer (`bouton.c`)	pour renvoyer : débrancher puis rebrancher le Pico
numéro d'envoi et anti-doublon	chaque nouvel envoi **s'ajoute** au précédent : réimporter `creer_base.sql` pour vider la table
vérification des lignes, réponses JSON, statistiques	une ligne mal formée provoque une erreur PHP ; la page n'affiche plus moyenne et écart-type

Si la LED ne s'allume pas : Apache et MySQL sont-ils démarrés dans XAMPP ? `SERVEUR_IP` dans `config.h` est-elle bien l'IP du PC (`ipconfig`) ? Le PC est-il sur le Wi-Fi « wifirpi » ?
Tester le serveur sans le Pico (PowerShell, dans `docs/`) : `curl.exe --data-binary "@paquet_test.csv" "http://localhost/cpas/reception.php"` doit répondre `OK`.
La version complète (robuste) est conservée dans `CPAS_ENVOI_SERVEUR_ORIGINAL_ROBUSTE`.